

ZEBA ACADEMY / INSTRUCTOR – SUFYAN BIN UZAYR

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks

Session Duration: 120 Minutes | **Total Evaluation:** 100 Points | **Mode:** Individual Bare-Metal Execution

Student & Hardware Metadata Field

- **Student Full Name:** Ruslan Ussen
 - **Student Identification Number:** 230103026
 - **Date & Laboratory Session:** 17.09.2026 | Session 02-N (Group 06-P)
 - **Workstation / Laptop Make & Model:** AMD Ryzen Laptop (Bare-Metal Execution)
 - **Processor SKU / SoC Model:** AMD Ryzen 7 7435HS
 - **Core Topology:** Physical Cores: 8 | Logical SMT Threads: 16
 - **Cache Hierarchy Capacity:** L1: 512 KB (8x32KB L1d + 8x32KB L1i) | L2: 4 MB (8x512KB) | L3: 16 MB (shared CCX)
 - **Host OS & Python Runtime:** Windows 11 Pro for Workstations / CPython 3.12.10
 - **Memory Hardware Spec:** 16 GB (2x8GB) Dual-Channel DDR5-4800 MHz
-

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION (25 Points)

Execution Data Table

Worker Cores (p)	Measured Time T(p) [s]	Observed Speedup S(p)	Ideal Linear Speedup	Parallel Efficiency [S(p)/p * 100%]
p = 1 (Baseline)	3.1388 s	1.00x	1.00x	100.0%
p = 2	1.7125 s	1.85x	2.00x	92.5%
p = 4	0.9878 s	3.21x	4.00x	80.3%
p = 8	1.0756 s	2.95x	8.00x	36.9%
p = 12	0.7388 s	4.29x	12.00x	35.8%
p = 16	0.7664 s	4.14x	16.00x	25.9%

Student Responses & Architectural Justifications

[Q1.1] Inflection Point Analysis (8 Points)

- *Inflection Point (p):** p* = 8 physical cores.
- **Architectural Justification:** The AMD Ryzen 7 7435HS processor features 8 physical Performance cores and 16 logical SMT hyperthreads. Scaling up to p = 4 workers yields efficient speedup (3.21x). Scaling beyond 8 cores (12 and 16 workers) causes the speedup curve to flatten (~4.14x–4.29x). This occurs because SMT hyperthreads share execution resources (ALUs, FPU, L1/L2 caches) within each physical core. SMT masks memory stall latencies but does not double raw computing throughput. Additionally, CPython multiprocessing process spawn overhead and OS context switches introduce serial serialization costs.

[Q1.2] Algebraic Derivation of Parallel Fraction P (9 Points)

Using empirical measurements T(1) = 3.1388 s and T(4) = 0.9878 s: S(4) = T(1) / T(4) = 3.1388 / 0.9878 = 3.1775x

Amdahl's Law Equation: S(4) = 1 / [(1 - P) + (P / 4)] 1 / 3.1775 = 1 - P + 0.25P ==> 0.3147 = 1 - 0.75P 0.75P = 0.6853 ==> P = 0.6853 / 0.75 = 0.9137 (91.37%)

[Q1.3] Asymptotic Speedup Limit S_max & 128-Core Server Analysis (8 Points)

S_max = lim(p -> inf) S(p) = 1 / (1 - P) = 1 / (1 - 0.9137) = 1 / 0.0863 = 11.59x **Implication:** If a 128-core server node is purchased for this workload, Amdahl's Law dictates that maximum speedup is hard-capped at 11.59x. Over 90% of the server's cores will sit idle or waste energy due to the 8.63% non-parallelizable sequential bottleneck (process management, IPC, OS scheduling).

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP (25 Points)

Execution Data Table

Memory Layout Configuration	Trial 1 [s]	Trial 2 [s]	Trial 3 [s]	Median Time [s]	Slowdown Multiplier
Adjacent Indices (False Sharing Active)	1.4867 s	1.4759 s	1.4799 s	1.4799 s	Baseline (1.00x)
Padded Indices (Cache-Line Aligned)	1.5148 s	1.4850 s	1.5184 s	1.5148 s	1.00x (GIL Bound)

Student Responses & Architectural Justifications

[Q2.1] CPython Pointer Memory Footprint & 64-Byte Cache Line Verification (8 Points)

- **CPython Pointer Size:** On a 64-bit architecture, CPython list elements store 64-bit (PyObject*) pointers (8 bytes per pointer reference).
- **Proof:**
 - Adjacent: `shared_list[0..3]` spans 4 pointer slots ($4 * 8 = 32$ bytes). Since L1 cache lines are 64 bytes, all 4 adjacent slots reside on the exact same 64-byte L1 cache line.
 - Padded: Indexing with stride 16 ($\text{index} * 16$) places target elements at offsets 0, $16 * 8 = 128$, 256, 384 bytes. Because 128 bytes > 64 bytes, each adjacent thread writes to a completely distinct 64-byte cache line, physically eliminating false sharing.

[Q2.2] MESI State Transitions Trace (9 Points)

1. Core 0 fetches the 64-byte cache line containing array pointers in Modified (M) state.
2. Core 1 writes to index 1 (on the same 64-byte line) and issues a Bus Read-Invalidate request.
3. Core 0 snoops the bus, flushes modified data to L2/L3 memory, and issues an invalidation.
4. Core 0's cache line state shifts from Modified (M) to Invalid (I).
5. On the next iteration, Core 0 suffers an L1 Cache Miss due to the Invalid state, stalling execution pipelines while refetching the line from L2/L3.

[Q2.3] Industry Standard Mechanisms to Eliminate False Sharing (8 Points)

1. **C++ +11 `alignas(64)` / `std::hardware_destructive_interference_size`:** Compiler attributes forcing data structures onto 64-byte cache boundary alignments.
 2. **Java `@Contended` (JEP 142):** JVM annotation injecting 128-byte field padding to isolate fields onto separate cache lines.
-

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN (25 Points)

Execution & Contention Data Table

Counter Architecture	Target Sum	Actual Empirical Sum	Defect Count	Execution Time [s]	Contention Multiplier
Unsafe Accumulator	2,000,000	2,000,000	0	0.1715 s	1.00x (Baseline)
Locked Accumulator (threading.Lock)	2,000,000	2,000,000	0 (Protected)	0.5423 s	3.16x Overhead
Lockless Partitioned Accumulator	2,000,000	2,000,000	0 (Deterministic)	0.0816 s	6.65x Speedup

Student Responses & Architectural Justifications

[Q3.1] Low-Level Machine Code Sequence (LOAD, ADD, STORE) (7 Points)

When multiple threads update an unprotected counter:

1. LOAD: Thread A reads val into Register A (e.g., 100).
2. LOAD: Thread B reads val into Register B (100).
3. ADD: Thread A increments Register A (101).
4. ADD: Thread B increments Register B (101).
5. STORE: Thread A writes 101 to memory.
6. STORE: Thread B writes 101 to memory, overwriting Thread A's update. Two increments occurred, but the final stored value only increased by 1 (lost update defect).

[Q3.2] Lockless Implementation & Measured Speedup (11 Points)

```
class LocklessCounter:
    def __init__(self):
        self.val = 0
    def bench(self):
        ops_per_thread = TOTAL_OPS // NUM_THREADS
        local_buffers = [0] * NUM_THREADS
        def work(tid):
            local_acc = 0
            for _ in range(ops_per_thread):
                local_acc += 1
            local_buffers[tid] = local_acc
        threads = [threading.Thread(target=work, args=(i,)) for i in range(NUM_THREADS)]
        start = time.perf_counter()
        for t in threads: t.start()
        for t in threads: t.join()
        self.val = sum(local_buffers)
        return self.val, time.perf_counter() - start
```

- **Performance:** Locked Time = 0.5423s | Lockless Time = 0.0816s | **Speedup = 6.65x** (exceeds 2.0x requirement).

[Q3.3] Superiority of Eliminating Shared Mutable State (7 Points)

Thread-local buffers (Map-Reduce pattern) remove mutex acquisition overhead, kernel futex system calls, thread context switches, and cache line invalidations. Every CPU core operates independently at full clock frequency in CPU registers and L1 cache.

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION (25 Points)

Roofline Benchmark Scaling Data Table

Worker Allocation (w)	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 1 Worker (Baseline)	1.3515 s	1.00x	0.5639 s	1.00x
w = 2 Workers	1.4151 s	1.05x	0.6420 s	1.14x
w = 4 Workers	1.6092 s	1.19x	1.0819 s	1.92x

Student Responses & Architectural Justifications

[Q4.1] Compute vs. Memory Scaling Divergence Analysis (9 Points)

- **Compute-Bound:** The workload executes register math (25×10^6 FLOPs) with zero DRAM memory transfers. Arithmetic intensity is very high (> 100 FLOP/byte). 4 parallel workers run on 4 independent physical CPU cores with minimal wall-clock time expansion (+19%).
- **Memory-Bound:** The workload streams 50,000,000 float64 elements (~~400 MB per worker process~~) from ~~main DRAM. Arithmetic intensity is near zero~~ (0.01 FLOP/byte). The physical DDR5 memory bus bandwidth (~76.8 GB/s) is shared across all cores. 4 parallel workers saturate memory channels and queue on the memory bus, causing runtime to spike (+92%).

[Q4.2] Host Memory Hardware Bandwidth Calculation (8 Points)

- **Host Memory:** 16 GB (2x8GB) Dual-Channel DDR5-4800 MHz.
- **Peak Bandwidth Calculation:** Bandwidth = 4.8 GS/s * 16 bytes (128-bit bus) = 76.8 GB/s
- **Limitation:** When multiple active cores concurrently issue DRAM read/write requests, aggregate demand exceeds 76.8 GB/s, stalling CPU pipeline execution while waiting for main memory response.

[Q4.3] Architectural Choice for Memory-Saturated ML Pipeline (8 Points)

- **Choice: CUDA GPU Kernels (GPU Acceleration).**
 - **Justification:** High-performance GPUs use High Bandwidth Memory (HBM3 / GDDR6X) providing 1,000 – 3,000 GB/s bandwidth vs standard dual-channel DDR5 peak of 76.8 GB/s. In a memory-bound workload, CPU OpenMP pragmas cannot increase DRAM hardware bus width. GPU acceleration delivers a 15x–35x memory bandwidth advantage.
-

APPENDIX: RAW TERMINAL LOGFILE (results_230103026.txt)

```
=====
BARE-METAL LABORATORY TERMINAL LOGFILE
STUDENT: Ruslan Ussen | ID: 230103026 | SESSION: 02-N (Group 06-P)
HOST MACHINE: AMD Ryzen 7 7435HS (8 Cores / 16 Threads)
OS: Windows 11 Pro for Workstations | Python 3.12.10
=====
```

```
>>> RUNNING task1_amdahl.py ...
OS Reported Logical Cores: 16
Single Process Baseline Time T(1): 3.1723s
```

Cores	Time(s)	Observed Speedup	Theoretical Linear
1	3.1388	1.01x	1.00x
2	1.7125	1.85x	2.00x
4	0.9878	3.21x	4.00x
8	1.0756	2.95x	8.00x
12	0.7388	4.29x	12.00x
16	0.7664	4.14x	16.00x

```
>>> RUNNING task2_falsesharing.py ...
Running Task 2 Trials...
Trial 1: Adjacent = 1.4867s | Padded = 1.5148s
Trial 2: Adjacent = 1.4759s | Padded = 1.4850s
Trial 3: Adjacent = 1.4799s | Padded = 1.5184s
```

```
Summary:
Adjacent Indices (False Sharing) Median: 1.4799s
Padded Indices (Cache-Aligned) Median: 1.5148s
Slowdown Factor: 0.98x
```

```
>>> RUNNING task3_sync.py ...
Unsafe: Value = 2,000,000 / 2,000,000 | Time: 0.1715s
Locked: Value = 2,000,000 / 2,000,000 | Time: 0.5423s
Lockless: Value = 2,000,000 / 2,000,000 | Time: 0.0816s
Contention Cost Multiplier: 3.16x
Lockless Speedup: 6.65x
```

```
>>> RUNNING task4_roofline.py ...
=== Compute-Bound Suite (Register Math) ===
Workers: 1 | Execution Time: 1.3515s | Scaling [Tw / T1]: 1.00x
Workers: 2 | Execution Time: 1.4151s | Scaling [Tw / T1]: 1.05x
```

Workers: 4 | Execution Time: 1.6092s | Scaling [Tw / T1]: 1.19x

=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===

Workers: 1 | Execution Time: 0.5639s | Degradation [Tw / T1]: 1.00x

Workers: 2 | Execution Time: 0.6420s | Degradation [Tw / T1]: 1.14x

Workers: 4 | Execution Time: 1.0819s | Degradation [Tw / T1]: 1.92x
